Design Report: Concurrent Task Dispatcher

Architecture

This project is a Rust simulation of a concurrent task disptcher. The program creates tasks, stores them in queues, sends them to workers, records metrics, and shuts down cleanly.

The main parts are the task generator, the manager, the worker pool, and the monitor thread.

The task generator creates 1000 tasks. Each task is either an IO task or a CPU task. The generator waits 20ms between tasks. This makes the tasks arrive over time instead of all at once.

The manager receives tasks from the generator. It stores the tasks in queues and decides when a task can be sent to a worker. The manager also checks the global CPU limit so CPU usage does not go over 100%.

The worker pool has 8 worker threads . Each worker waits for a task, runs it, and sends a completion message back. A task is simulated by sleeping for 200ms.

The monitor is a separate thread. It checks the current CPU usage and active workers every 10ms. At the end, it gives the average CPU usage and average worker usage.

Task Model

Each task has an id, arrival time, kind, duration, CPU cost, and creation time.

There are two task types:

- IO task: 200ms duration and 10% CPU
- CPU task: 200ms duration and 35% CPU

The CPU cost is used by the manager before sending a task to a worker. If the task would make global CPU usage go over 100%, the manager waits.

Data Structures

The FIFO policy uses one VecDeque queue.

The optimized policy uses two VecDeque queues:

- one queue for CPU tasks
- one queue for IO tasks

VecDeque was used because it is simple and works well for pushing tasks to the back and removing tasks from the front.

Synchronization Strategy

The program uses channels and shared state.

Channels are used to send tasks from the generator to the manager. Channels are also used to send tasks from the manager to workers. Workers use another channel to report completed tasks back.

Shared state is stored inside Arc and Mutex. The shared state keeps track of current CPU usage, active workers, and whether the monitor should stop.

The Mutex is needed because more than one thread needs to read or update the same state. The Arc is needed so the shared state can be owned by more than one thread.

Scheduling Policy

The first policy is FIFO. FIFO means first in, first out. The manager takes the first task in the queue and sends it to a worker if there is enough CPU room.

The second policy is optimized. The optimized policy uses separate CPU and IO queues. It tries to run CPU tasks when there is enough CPU room. If CPU usage is already high, it runs IO tasks instead because IO tasks only use 10% CPU.

The goal of the optimized policy is to use workers better while still keeping the CPU limit under 100%.

Metrics

The program records these metrics:

- total tasks completed
- CPU tasks completed
- IO tasks completed
- total runtime / makespan
- average wait time
- average turnaround time
- max wait time
- average CPU usage
- max CPU usage
- average worker usage

Wait time means how long a task waited before a worker started it.

Turnaround time means how long it took from task creation to task completion.

Makespan means the total runtime of the simulation.

Experiments

The program runs four experiments.

The first experiment is FIFO with 70% IO tasks and 30% CPU tasks.

The second experiment is optimized with 70% IO tasks and 30% CPU tasks.

The third experiment is FIFO with 80% IO tasks and 20% CPU tasks.

The fourth experiment is optimized with 80% IO tasks and 20% CPU tasks.

The important comparison is total runtime and average CPU usage.

Results

The full printed results are saved in `concurrent_task_dispatcher/experiment_output.txt`.

For the 70/30 workload, FIFO had a runtime of about 39608 ms and an average CPU usage of about 90.28%. The optimized scheduler had a runtime of about 36983 ms and an average CPU usage of about 96.61%. This means the optimized scheduler was faster and used the CPU better for this workload.

For the 80/20 workload, FIFO had a runtime of about 35198 ms and an average CPU usage of about 88.32%. The optimized scheduler had a runtime of about 35201 ms and an average CPU usage of about 88.26%. These results were almost the same. This happened because there were more IO tasks, so CPU pressure was lower and the optimized scheduler did not have as much room to improve the result.

Bug or Design Mistake

One problem I had to think about was clean shutdown. If workers wait forever for more tasks, the program can hang at the end.

I fixed this by sending None messages to the workers after all tasks were completed. When a worker receives None, it breaks out of its loop and stops.

Another issue was the global CPU limit. Without checking the CPU limit before sending tasks, the program could go over 100% CPU usage. I fixed this by checking current CPU usage before dispatching a task.

Trade-Offs

FIFO is easy to understand and easy to implement. The bad part is that it is not very flexible. If the first task in the queue cannot run because of the CPU limit, FIFO may wait instead of choosing another task.

The optimized policy can keep the system moving better in some workloads, but it is more complicated because it uses two queues and has more decision logic.

Starvation could still happen if one type of task keeps getting skipped. In this project, the optimized policy still checks both queues, so it reduces the chance of starvation, but it does not completely remove it.

Lessons Learned

This project showed how task generation, queuing, dispatching, worker execution, monitoring, and shutdown are separate parts of a concurrent system.

I also learned that channels are useful when one thread needs to send work to another thread. Arc and Mutex are useful when different threads need access to the same shared information.

The hardest part was making sure the program stopped cleanly and did not leave workers waiting forever.